Task 1 — Data Pipeline Architecture

GCP Analytics Platform Design | Stockbit/Bibit

1. Assumptions

The following assumptions scope this architecture design. All tool selections and tradeoffs are grounded in these constraints.

Industry & Business Context

- Fintech platform (stock trading + mutual fund/reksa dana) operating in Indonesia
- Primary users: retail investors via mobile/web app
- Business-critical operations: order execution, portfolio valuation, payment settlement

Data Volume & Scale

- Historical data: ~5 TB accumulated
- Daily ingestion: ~50 GB/day across all sources
- Peak traffic: market hours (09:00–16:00 WIB weekdays), up to 10x normal volume during high-volatility events

Latency Requirements

- **Market price feeds, order status, fraud/anomaly alerts:** Real-time (< 1 min)
- **User transaction events, app clickstream — micro-batch:** Near real-time (1–15 min)
- **Regulatory reports, portfolio reconciliation, partner settlement — overnight:** Batch (daily)

Data Consumers

- Internal: data analysts, data scientists, product & business teams
- External-facing: user portfolio summary served via API to mobile app
- Concurrent BI dashboard users: ~50–100 internal

Compliance & Governance

- OJK-regulated: data residency must stay within Indonesia — GCP region asia-southeast2 (Jakarta)
- Financial transaction data retention: minimum 5 years
- PII masking required: NIK, phone number, email address

Budget

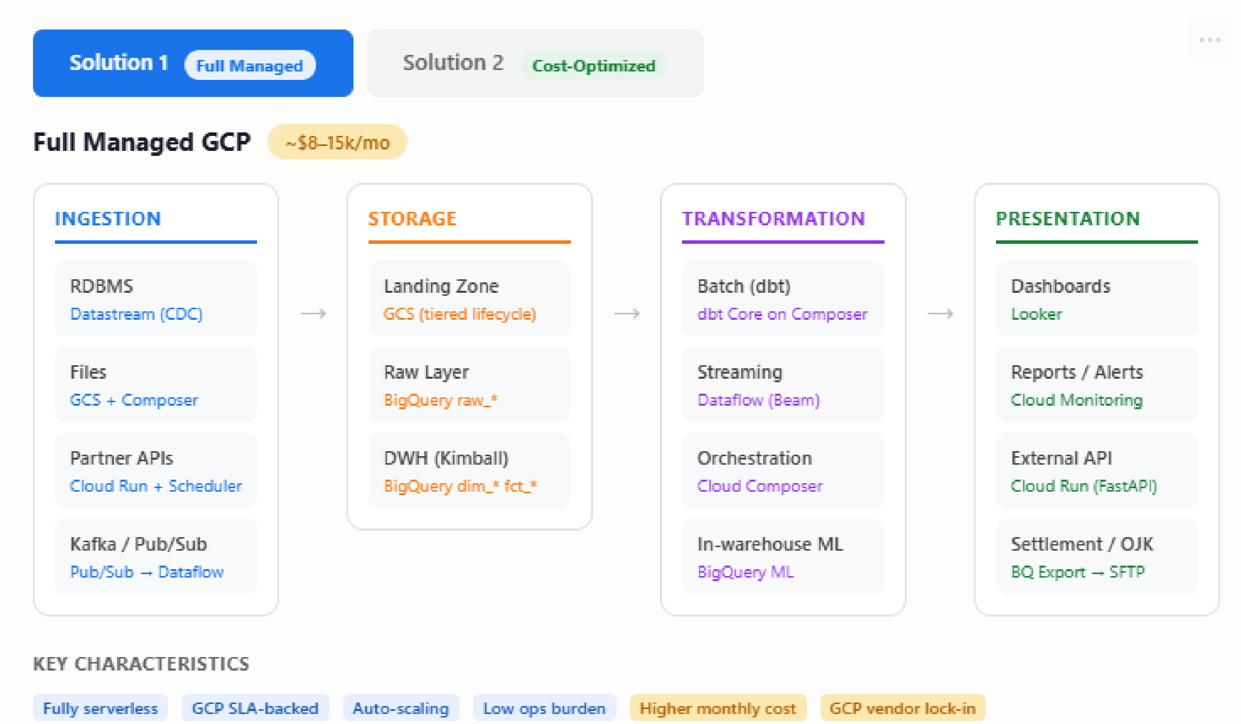
- Mid-range budget, optimized for cost-efficiency without sacrificing reliability
- Prefer managed services: tradeoff of higher GCP bill vs lower engineering ops overhead
- BigQuery pricing: on-demand for ad-hoc queries, flat-rate slots for scheduled production workloads
- GCS tiering: Hot (0–90 days) → Nearline (90 days–1 year) → Coldline (> 1 year)

2. Architecture Overview

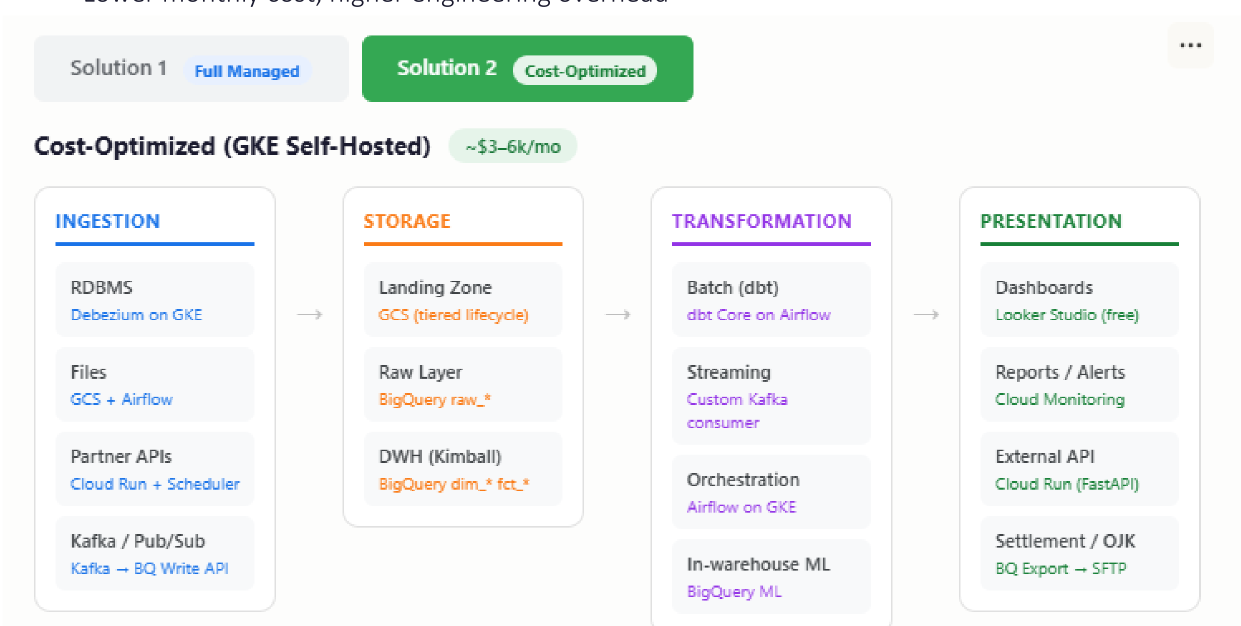
The platform is structured into four layers: Ingestion, Storage, Transformation, and Presentation. Data Quality Control (DQC) operates as a cross-cutting concern with fundamentally different implementations for batch and streaming paths.

Two solution options are provided based on budget and operational constraints:

- Solution 1 — Full Managed: leverages GCP-managed services throughout. Higher monthly cost, minimal ops burden



- Solution 2 — Cost-Optimized: replaces key managed services with self-hosted equivalents on GKE. Lower monthly cost, higher engineering overhead



3. Layer 1 — Ingestion

Source	Solution 1 (Full Managed)	Solution 2 (Cost-Optimized)	Justification
RDBMS	Datastream (CDC)	Debezium on GKE	CDC captures row-level changes with minimal disruption to source DB. Datastream is fully managed with direct BQ/GCS output; Debezium is more flexible but requires GKE management.
Files (CSV, Parquet, Avro, Sheet)	GCS + Cloud Composer	GCS + Airflow on GKE	Files dropped to GCS trigger Composer/Airflow DAGs for processing. Auditable, cost-efficient, no streaming overhead needed.
Partner APIs	Cloud Run + Cloud Scheduler	Cloud Run + Cloud Scheduler	Containerized Python scripts pull APIs on schedule and push to GCS or Pub/Sub. Stateless, scale-to-zero = minimal cost. Same across both solutions.
Kafka	Pub/Sub connector → Dataflow	Direct Kafka consumer → BQ Write API	Sol 1 bridges Kafka to Pub/Sub for native GCP integration, then Dataflow handles stream processing with auto-scale. Sol 2 writes directly to BQ Storage Write API via custom consumer — cheaper but needs manual tuning.
Pub/Sub (native)	Dataflow (Apache Beam)	Dataflow (Apache Beam)	Directly consumes Pub/Sub topics. BQ Storage Write API used over streaming inserts for lower cost.

4. Layer 2 — Storage

GCS — Raw Landing Zone

All ingested data lands in GCS first, partitioned by source and date. This enables data replayability — if a transformation bug is found, data can be re-processed from raw without re-ingesting from the source system.

- Hot (0–90 days): active pipeline data, frequent access
- Nearline (90 days–1 year): historical queries, infrequent access
- Coldline (> 1 year): compliance retention, rare access — lowest cost

BigQuery — Data Warehouse (Kimball Model)

The warehouse follows the Kimball dimensional modeling approach with a star schema. Two dataset layers are maintained:

Dataset	Contents	Materialization
raw_*	As-landed data from GCS/Datastream. Minimal transformation, append-only.	BigQuery tables (partitioned by ingestion date)
dim_*	Dimension tables: customers, products, sellers, dates. SCD Type 2 via MERGE for slowly changing attributes.	BigQuery tables (MERGE/upsert)
fct_*	Fact tables: orders, payments, order items, reviews. Grain defined per business process.	BigQuery tables (partitioned by event date)

Note: Staging logic (column renaming, type casting, deduplication) is implemented as ephemeral dbt models — not materialized as physical tables in BigQuery. This keeps the warehouse clean while preserving transformation logic in version control.

Dead Letter Storage

- Batch: GCS dlq_* prefix — invalid records written here on DQC failure
- Stream (Sol 1): Pub/Sub Dead Letter Queue — failed records routed for manual review and replay
- Stream (Sol 2): Kafka Dead Letter Topic (DLT) — native to Kafka ecosystem

5. Layer 3 — Transformation

Tool	Use Case	Solution	Justification
dbt Core	Batch transformations: raw_* → dim_* / fct_*	Both	SQL-based, version-controlled in Git, built-in testing and documentation, lineage graph. Industry standard for BigQuery transformations. Ephemeral staging models avoid physical stg_* tables.
Dataflow (Apache Beam)	Stream transformations: real-time event processing	Both	Managed, auto-scales during market hours peak (up to 10x). No cluster management. Handles DQC inline per record before writing to BQ.
Cloud Composer	Orchestration & DAG scheduling	Sol 1 only	Managed Airflow. Handles batch DAG dependencies, retry logic, and alerting. Natively integrates with GCP services.
Airflow on GKE	Orchestration & DAG scheduling	Sol 2 only	Self-hosted Airflow. Lower cost than Composer but requires GKE cluster management and manual upgrades.
BigQuery ML	In-warehouse ML (fraud scoring, churn)	Both	Lightweight ML without moving data outside the warehouse. Suitable for models that don't require Vertex AI complexity.

6. Data Quality Control (DQC)

DQC is a cross-cutting concern. The approach is fundamentally different for batch and streaming paths due to their latency characteristics.

Batch DQC — Synchronous Gate

In the batch path, DQC acts as a hard gate between storage and transformation. The pipeline can be halted on failure without impacting end users.

Step	Action	Tool
1. Load	Data lands in raw_* from GCS/Datastream	Datastream / GCS
2. DQC check	Run validation: not null, unique, accepted values, referential integrity, statistical checks	dbt tests + Great Expectations
3a. PASS	Proceed with dbt transformations into dim_* and fct_*	dbt Core
3b. FAIL	Halt pipeline. Route failed records to GCS dlq_*. Trigger alert.	Cloud Monitoring + Composer/Airflow
4. Review	Data engineer investigates, fixes upstream issue, replays from raw_*	Manual

Reconciliation: For fintech, an additional reconciliation DAG runs after batch — verifying transaction counts in fct_orders match settlement files sent to SFTP partners.

Stream DQC — Async In-Band

In the streaming path, halting the pipeline to wait for validation is not feasible. Valid records must not be blocked by invalid ones. DQC runs inline per-record inside the Dataflow job.

Step	Action	Sol 1 Tool	Sol 2 Tool
1. Consume	Read events from Pub/Sub or Kafka	Pub/Sub	Kafka
2. DQC per record	Validate schema, required fields, value ranges within Dataflow transform	Dataflow (Beam)	Custom Kafka consumer
3a. PASS	Write valid records to BigQuery via Storage Write API	BQ Storage Write API	BQ Storage Write API
3b. FAIL	Route invalid records to Dead Letter destination (non-blocking)	Pub/Sub DLQ	Kafka Dead Letter Topic
4. Alert	Spike in DLQ/DLT triggers alert	Cloud Monitoring	Cloud Monitoring
5. Replay	After fix, replay DLQ/DLT back into main pipeline	Pub/Sub replay	Kafka consumer replay

7. Layer 4 — Presentation

Output	Solution 1	Solution 2	Justification
Internal Dashboards	Looker	Looker Studio (free)/Metabase	Sol 1: Looker provides governed semantic layer, row-level security critical for financial data, embedded analytics. Sol 2: Looker Studio/Metabas is free with native BQ connector, sufficient for 50-100 internal users.
Pipeline & Data Alerts	Cloud Monitoring + Pub/Sub	Cloud Monitoring + Pub/Sub	Pipeline failure alerts, data freshness SLA breaches, DLQ spike notifications. Routes to Slack/email. Same across both solutions.
Business Alerts	Looker scheduled alerts	BigQuery scheduled queries	Threshold-based alerts for anomalies: transaction spikes, user activity drops.
External API (app-facing)	Cloud Run (FastAPI)	Cloud Run (FastAPI)	Serves portfolio summary to mobile app. Queries fct_* and dim_* from BigQuery mart. Stateless, scale-to-zero = cost-efficient.
Settlement Files (CSV)	BQ Export → GCS → Cloud Run SFTP	BQ Export → GCS → Cloud Run SFTP	Scheduled BQ query exports to GCS, Cloud Run delivers to partner SFTP endpoints via Python paramiko. Triggered by Cloud Scheduler after overnight batch completes.
Regulatory Reports (XML, CSV)	BQ → Cloud Run → SFTP/email	BQ → Cloud Run → SFTP/email	Generate OJK-required XML/CSV reports from BigQuery. Cloud Scheduler triggers after batch. Same approach across both solutions.
Analytical / ML Results	BigQuery ML + Vertex AI	BigQuery ML	Sol 1: Vertex AI for complex models (fraud detection, portfolio risk). Sol 2: BQML only for lightweight in-warehouse scoring.

8. Cross-Cutting: Governance & Security

These controls apply to both solutions and are non-negotiable given OJK compliance requirements.

Concern	Tool	Detail
PII Masking	Cloud DLP	Auto-detect and mask NIK, phone, email before data enters the warehouse from raw_* layer.
Column-Level Security	BigQuery column-level security	Restrict access to sensitive columns (e.g., account numbers) per IAM role. Analysts see masked values by default.
Encryption	CMEK (Customer-Managed Encryption Keys)	Applied to GCS buckets and BigQuery datasets. Required for financial data under OJK guidelines.
Data Lineage	dbt docs + Dataplex	dbt generates lineage graphs for all batch transformations. Dataplex provides catalog and lineage across GCS and BQ assets.
Audit Logging	Cloud Audit Logs	All data access and pipeline operations logged. Retention aligned with 5-year compliance requirement.

9. Solution Comparison & Recommendation

Dimension	Solution 1 — Full Managed	Solution 2 — Cost-Optimized
CDC	Datastream — managed, zero-disruption	Debezium on GKE — flexible, self-managed
Streaming	Dataflow — auto-scale, SLA-backed	Custom Kafka consumer — cheaper, needs tuning
Stream DLQ	Pub/Sub DLQ — managed replay	Kafka Dead Letter Topic — ops-heavy
Batch DQC	dbt tests + Great Expectations on Composer	dbt tests + Great Expectations on Airflow
Orchestration	Cloud Composer — fully managed Airflow	Self-hosted Airflow on GKE
BI	Looker — governed metrics, row-level security	Looker Studio — free, limited governance
Estimated Cost	~$8,000–$15,000 / month	~$3,000–$6,000 / month
Ops Burden	Low — GCP handles infra, upgrades, scaling	High — team manages GKE, Debezium, Airflow
Time to Production	Days	Weeks
Scalability	Auto-managed, elastic	Manual tuning required
Vendor Lock-in	High (GCP-native)	Lower (more portable)
Best For	Small eng team, strict SLA, OJK compliance priority	Cost pressure, larger eng team, portability needed

Recommendation

For a fintech platform operating under OJK compliance with strict SLA requirements and a small-to-mid engineering team, Solution 1 (Full Managed) is recommended. The higher GCP bill is justified by: lower risk of pipeline failures impacting financial operations, faster time-to-production, and reduced engineering effort spent on infrastructure maintenance rather than business-value work.

Solution 2 becomes viable when: the engineering team has dedicated infrastructure capacity, cost reduction is a board-level priority, or the platform needs to remain portable across cloud providers.